DressMe · BO1 & BO2 agents ESPRIT 2026

BO1 Personalised recommendations and BO2 Time optimisation (Racem Njeh). Every idea is run by an LLM agent; one orchestrator agent per BO controls its idea agents; the shared modules (M2, M3, M4, M7, M9, M17) are their tools. Everything below runs in the prototype and was tested.

1. Architecture: three levels of agents

Level 3 · BO orchestrators (LLM routing + guardrails, cross-BO delegation)
personalised_recommendations (BO1) ⇄ time_optimisation (BO2) ← user / BO8 orchestrator / other BO agents

Level 2 · Idea agents (each: own LLM picks action + arguments → workflow → LLM answer)
BO1: daily_outfit · rotation · forgotten · capsule · seasonal_refresh · avatar_preview · gap_filling · style_insights
BO2: daily_plan · swap · laundry · event_looks · local_shops · photo_piece · declutter
BO1+BO2: travel (travel planner + packing)
16 idea agents covering 26 ideas · BO8 protocol: AgentCall {user_id, session_id, language, message, payload} → AgentResult {agent, result, question, next_actions}

Level 1 · Tools = module agents (structured actions, no LLM) + models
M2 recommendation (OutfitTransformer + FashionCLIP) · M3 wear log · M4 scheduler/calendar · M17 capsule (ILP)
M9 3D try-on (Blender cloth) · M7 local catalogue (BO6) · Open-Meteo weather · FashionCLIP zero-shot
called through the agent bus: shared.agent_bus.ask(name, action, payload) — the same message over HTTP: POST /agents/{name}

Figure 1 · An outer agent only talks to the level below through messages; the LLM decides at levels 2 and 3, the tools do the exact work.

2. LLM used

Choice	**Qwen2.5 3B Instruct** (Alibaba, open weights) served locally by **Ollama** (`qwen2.5:3b` , ~1.9 GB).
Why	Runs offline on the team's laptop GPU (RTX 2050, 4 GB) — no API key, no cost, no user data sent to a cloud; supports **tool calling** and a **JSON output mode** (constrained decoding), which the routing needs; good in English and French. Measured: 3–9 s per LLM call.
Used for	(1) orchestrator routing: which idea agent; (2) idea-agent routing: which action + arguments, from the user's sentence; (3) writing the answer from the tool results (grounded); (4) extracting a preference ("I hate yellow").
Swap	One setting: `LLM_LOCAL_MODEL` (e.g. `qwen2.5:7b`) or `shared/llm.py` (Claude / OpenAI) for production. Code: `backend/shared/local_llm.py` .

3. Memory

Type	What	Where / how
Short-term · conversation	The turns of a chat session; the last 4–6 are replayed to the routing and answer LLMs (follow-ups like "why this outfit?").	`shared/agent_memory.py` · JSON per user / session in `backend/.agent_memory/`
Short-term · session blackboard	Shared working memory of all agents of a session: last outfits, last day, last occasion, capsule, last photo. "Change the shoes" (swap agent, BO2) works on the outfit chosen by daily_outfit (BO1).	`idea_agent.get_state / update_state` (`ctx.state`, `ctx.remember()`)
Long-term · taste profile	Likes / dislikes / notes the user stated; injected in every prompt; dislikes are applied as hard filters (excluded pieces).	`agent_memory.profile / remember`
Episodic · what happened	What was worn and when, laundry / lent status, posted looks — the facts behind rotation, forgotten pieces, drift, lookbook, declutter.	M3 database (SQLite / Postgres), read through the M3 agent
Semantic · the wardrobe	Every piece with its FashionCLIP vector (image + text); retrieved for each request.	M9 3D wardrobe (user's photos) or M2 demo wardrobes

4. Pipeline of one request

```
message ─► [BO orchestrator] memory · guardrail rule OR LLM routing (JSON) → idea agent
        ─► [idea agent]      memory · LLM routing (JSON): action + arguments (rule may lock the action)
                             workflow: wardrobe (RAG) → tools (M2/M3/M4/M7/M9/M17) → session blackboard
        ─► [BO orchestrator] LLM answer from the FACTS (recommended outfit first, numbers copied) ─► reply
                                                                                  + data (outfits, ids, 3D link) + steps
```

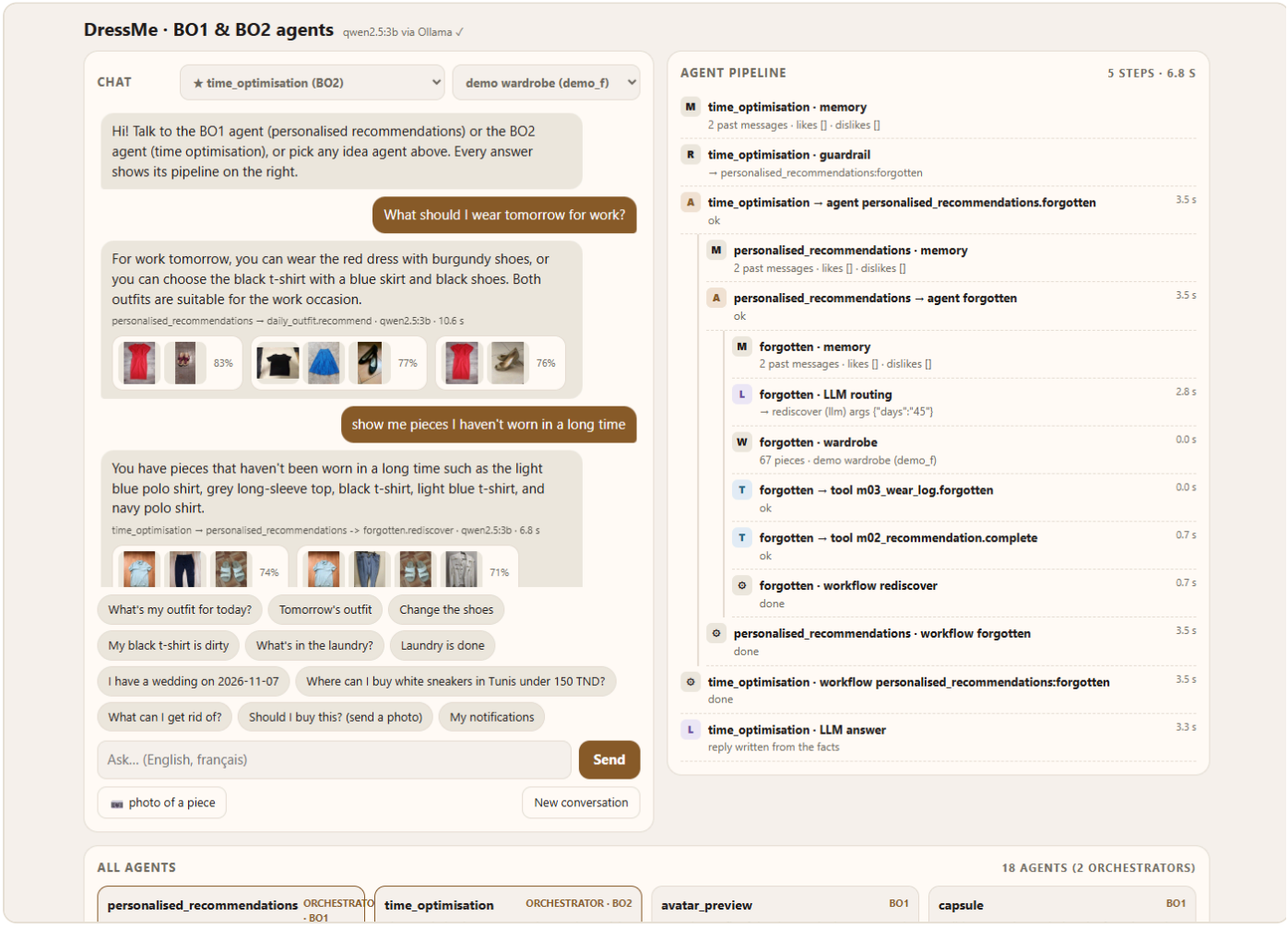


Figure 2 · Real run in the test web app (/bo/demo). "Show me pieces I haven't worn in a long time" sent to the **BO2** agent: guardrail → BO1 agent → *forgotten* agent (its LLM picks `rediscover`, days=45) → tools M3 forgotten + M2 complete → answer. Each line = one step with its time.

4b. When the LLM runs, and when it is skipped

Yes: every agent has an LLM (Qwen2.5 3B via Ollama). The LLM is the agent's **brain for language**: it understands a sentence and explains the result. It is **not** the part that picks the clothes: that is always the tools (OutfitTransformer in M2, the optimiser in M17, the M3 history…). So when the request is already precise (a button in the app, or another agent asking with a structured call), the agent **skips the LLM**: the result is the same, and it takes under a second instead of 6–20 s.

Who calls / how	Example	LLM calls	Why
User sentence → BO agent (orchestrator)	"What should I wear tomorrow for work?" → personalised_recommendations	**3**: choose agent · choose action + args · write answer	Free text: the agent has to understand it and answer in words.
User sentence, keyword rule matches	"my black t-shirt is dirty" → time_optimisation	**2**: the rule picks the laundry agent (LLM skipped), the LLM still extracts "black t-shirt", then writes the answer	Guardrail: a 3B model sometimes picks the wrong agent; a sure keyword is faster and safer.
User sentence → idea agent directly	"change the shoes" → swap	**2**: choose action + args · write answer	No orchestrator step needed.
Button in the app	"Change shoes" → `{"action":"swap","args":{"slot":"shoes"}}`	**0**	The button already says what to do; the screen shows the pieces, no sentence needed.
Agent → agent (inside a workflow)	gap_filling → local_shops; BO2 agent → BO1 agent → forgotten	**0** in the called agent	The caller sends the action; only the agent that talks to the user writes a reply (`reply:false` below it).
Agent → tool (M2, M3, M4, M7, M9, M17)	swap → M2 swap	**0**, never	Tools are exact code and models, not LLM agents.

In the code: `IdeaAgent.run` calls `route()` (LLM) only when no `payload.action` is given, and `write()` (LLM) only when `payload.reply` is true (default: true for a sentence, false for a structured call). Every LLM call appears in `result.steps` as `llm_route` / `llm_answer` with its time, so the demo shows when it ran.

5. AI techniques used

Technique	In the prototype
LLM agents with tool use	Each idea agent / orchestrator: the LLM selects a tool (action / agent) and its arguments, the code executes it, results go back to the LLM (ReAct-style loop, bounded).
Advanced prompt engineering	Role prompt per agent built from its card (role, actions, parameters), **JSON mode** (constrained output) for routing, explicit output rules, the user's language enforced as the last instruction, few-shot step examples without real piece names (a 3B model copies examples).
RAG (retrieval-augmented generation)	Before acting, the agent retrieves the user's wardrobe (FashionCLIP vectors; the pieces closest to the request), their wear history and laundry (M3), their calendar (M4) and their taste profile; the answer LLM sees only these retrieved facts.
Grounding / hallucination control	The answer is written from FACTS only: the recommended outfit first, one alternative, no ids; every number copied, never computed. Found by testing (the model mixed pieces and invented a cost per wear).
Guardrails (hybrid routing)	Keyword rules lock the agent / action when the wording is unambiguous ("dirty" → laundry, "haven't worn" → forgotten), the LLM still extracts the arguments; invalid LLM choices fall back to a default. Raised routing from 4/7 to 9/9 on the test conversation.
Pretrained models, no fine-tuning	OutfitTransformer (outfit compatibility, AUC 0.95), FashionCLIP (vectors, zero-shot type / colour of a photo), BiRefNet (background removal), Qwen2.5 (LLM). Project rule: nothing is trained; fine-tuning was not needed.
Optimisation & simulation	Integer linear programming (capsule, M17), greedy set reuse (packing), Blender cloth physics (3D garments, M9).

6. The agents, one by one

Input of every agent = `AgentCall`. A sentence in `message` lets the agent's LLM choose; or `payload = {"action": ..., "args": {...}}` to call an action directly (other agents, no LLM). Output = `AgentResult` with `result.reply`, `result.data`, `result.steps`, `question`, `next_actions`. Code: `backend/bo/bo1_recommendations/agents.py`, `backend/bo/bo2_time/agents.py`.

BO1 · Personalised recommendations

daily_outfit

Daily outfit + Remix · Outfit builder · "Why this outfit"

Actions: `recommend(day, occasion, mood)` · `remix()` · `build(pieces, occasion)` · `why(n)`
Tools: M4 calendar (occasion of the day) → M2 recommend / remix / complete / score; dislikes excluded; LLM explains the score breakdown (role of M8).
Workflow (recommend): day → M4 occasion → M2 top-3 outfits with weather → blackboard (last outfits) → answer. **Example:** "What should I wear tomorrow for work?" → work day, 22.7 °C → *red dress + burgundy shoes* (0.83), alternative black t-shirt + blue skirt + black shoes.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). For `why`, the LLM turns M2's score breakdown (colour, style, weather, rotation) into a sentence.

rotation

Rotation-aware penalty

Actions: `report()` · `fresh_outfits(days, occasion)`. **Tools:** M3 stats → M2 recommend with rotation window + pieces worn in the last N days excluded.
Example: report → most worn in 30 days: white shoes ×10, black shorts ×6; fresh outfits leave out 7 recently worn pieces.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given).

forgotten

Rediscover forgotten pieces

Action: `rediscover(days)`. **Tools:** M3 forgotten (available pieces only) → M2 complete around the most forgotten piece.
Example: light blue polo shirt (never worn) + navy pants + white shoes (0.74).

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given).

capsule

Capsule wardrobe mode

Actions: `build(size, season)` · `plan(days)`. **Tools:** M17 (integer linear programming over M2 scores), M4 occasions. **Example:** 8 pieces (autumn): grey long-sleeve top, grey t-shirt, white shirt, blue pants, yellow pants, grey shoes, white shoes, white jacket.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). The capsule itself is chosen by the optimiser (M17), never by the LLM.

seasonal_refresh

Seasonal wardrobe refresh

Action: `refresh(city)`. **Tools:** Open-Meteo forecast (next 7 days) → season · M3 forgotten (pieces to take out) · M2 looks with forgotten bonus. **Example:** mean 21.2 °C → spring/autumn, take out light blue polo shirt, grey long-sleeve top…

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given).

avatar_preview

3D avatar preview of the daily outfit

Action: `preview(animate)`. **Tools:** blackboard / M4 today → M9 outfit3d (cloth physics, cached) → link to the 3D viewer. Works on the user's 3D wardrobe (photos of their clothes).

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). The 3D render is Blender (M9); the LLM only adds the sentence.

gap_filling

Gap-filling suggestion (M6 not ready: analysis built here)

Action: `suggest(budget_tnd, city)`. **Tools:** wardrobe balance by category vs a target mix + missing neutral colour → estimate of new outfits → **local_shops** agent (M7). **Example:** missing slot bottom → black trousers → Moudda (Tunis) 69.9 TND.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). It calls the **local_shops** agent with a structured call, so local_shops does not use its LLM.

travel

Travel outfit planner (BO1) + Packing outfits for a trip (BO2)

Action: `plan(city, days, start, occasion)`. **Tools:** Open-Meteo weather of the destination per day → M2 6 candidates per day → greedy choice that reuses packed pieces → packing list. **Example:** "Je pars à Paris 3 jours" → 12 °C rain → 3 outfits, 6 pieces to pack (answer in French).

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). Writes the answer in the user's language ("Je pars à Paris" → French).

style_insights

Personal colour palette · Style drift detection · Favourite looks / lookbook

Actions: `palette()` · `drift()` · `lookbook()`. **Tools:** wardrobe colours × wear counts (M3) + colour harmony (complementary hue); M3 history: last 30 days vs 31–120; repeated / posted looks. **Example:** palette white 21 %, light blue 13 %, beige 12 %; rising burgundy, blue.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given).

BO2 · Time optimisation

daily_plan

Outfit of the day (cached) · Tomorrow's outfit the night before · Notification · Widget

Actions: `today` · `tomorrow` · `approve(day)` · `change(day)` · `wear` · `week` · `notifications` · `widget`. **Tools:** M4 (outfit computed at 05:00 / 20:00 and cached, approve / change / wear → M3, outbox, widget feed).
Example: widget → {"summary": "red dress + burgundy shoes", "temp_c": 21.8, "deep_link": "dressme://today"} in 0.0 s (cached).

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). The cached outfit (05:00 / 20:00) is computed without any LLM.

swap

One-tap swap of a single piece

Action: `swap(slot)`. **Tools:** blackboard (current outfit) or M4 today → M2 swap. **Example:** "change the shoes" → red dress + burgundy shoes → red dress + olive shoes.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given).

laundry

Laundry / unavailable UI

Actions: `status` · `put(pieces)` · `back(pieces)` · `lend(pieces, to)`. **Tools:** M3 status / laundry_done / states (the pieces are then excluded from every recommendation). **Example:** "my black t-shirt is dirty" → put → laundry.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). A keyword rule ("dirty", "sale") locks the action `put`; the LLM still extracts which pieces.

event_looks

Event countdown looks

Actions: `add(date, title, dress_code)` · `upcoming()`. **Tools:** M4 events (+ reminders) → M2 event looks with the event-day forecast. **Example:** "I have a wedding on 2026-11-07" → event added, 16 days left, look blue dress + black shoes + navy jacket.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). The LLM extracts the date and the title ("wedding on 7 Nov") from the sentence.

local_shops

Local shops shortlist for a missing piece

Action: `find(query)`. **Tools: M7 local catalogue (BO6, real)**: Tunisian shops, FashionCLIP + BM25 + attributes, TND budget, city. **Example:** "white sneakers in Tunis under 150 TND" → Manuella, Basket femme, 86.4 TND, match 0.95.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). The search is M7 (FashionCLIP + BM25); the LLM only turns the sentence into a query and writes the shortlist.

photo_piece

Complete the look from a photo · "Should I buy?" (M1 / M6 not ready: built here)

Actions: `complete_look(photo)` · `should_buy(photo, price_tnd)`. **Tools:** FashionCLIP image vector + zero-shot type / colour → the piece is added virtually to the wardrobe → M2 complete; duplicates by cosine > 0.9; verdict Buy (≥ 3 good outfits, no duplicate) / Think / Skip; cost per wear. **Example:** photo of a t-shirt, 45 TND → navy t-shirt, Buy, 5 good outfits, 0.9 TND per wear.

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given). Type and colour of the photo come from FashionCLIP zero-shot, not from the LLM.

declutter

Declutter suggestions (M1 not ready: built here)

Action: `suggest()`. **Tools:** M3 stats (never worn, not worn in 90 days) + near-duplicates by FashionCLIP (keep the most worn).

> **LLM:** sentence → its LLM picks the action + arguments (JSON), the tools run, then the LLM writes the answer from the results (2 LLM calls) · button or call from another agent → **skipped** (0 LLM calls, the action and arguments are given).

Orchestrators

personalised_recommendations (BO1) · time_optimisation (BO2 + "Quick natural-language request")

Input: any sentence (or `"<agent>.<action>"` for a direct call). **Workflow:** memory → guardrail rule, else LLM reads the idea agents' cards and returns `{"agent": ...}` (JSON) → the message goes to that idea agent (its LLM picks the action) → the orchestrator's LLM writes the answer from the facts. **Cross-BO:** a BO2 request about forgotten pieces goes BO2 agent → BO1 agent → forgotten agent (Figure 2), and "change the shoes" sent to BO1 goes to BO2's swap agent.

> **LLM:** sentence → the orchestrator's LLM picks the idea agent (**skipped** when a guardrail keyword matches), the idea agent's LLM picks the action, and the orchestrator's LLM writes the final answer (2–3 LLM calls in total) · structured call `"<agent>.<action>"` → **0 LLM calls**.



Figure 3 · The agents' cards in the test web app (ideas, tools, actions).

7. How the BO8 orchestrator reaches them

- The agents use BO8's own protocol: `shared/agent_card.py` (copied unchanged from `feat/bo8-orchestrator`): card (name, owner, role, skills, when_to_use, when_not_to_use, handoff), `AgentCall` in, `AgentResult` out with `question` and `next_actions`.
- BO8's planner can call **personalised_recommendations** and **time_optimisation** as two tools (one per BO, as BO8's AGENTS.md wants: the planner picks one agent, the agent decides); `next_actions` (e.g. "Show it on my avatar", "Find it in a shop") let it chain the next step.
- Same message over HTTP for agents in other services: `GET /agents` (cards), `POST /agents/{name}` (AgentCall), and `POST /bo/chat`.

```
POST /bo/chat
{"agent": "time_optimisation", "user_id": "demo_f", "session_id": "s1", "message": "change the shoes"}
→ {"agent": "time_optimisation", "question": null, "next_actions": [],
   "result": {"action": "swap", "reply": "…red dress with olive shoes…",
              "data": {"handled_by": "swap.swap", "slot": "shoes", "before": ["red dress", "burgundy shoes"],
                       "after": ["red dress", "olive shoes"], "alternatives": [["red dress", "black shoes"]]},
              "steps": [ … 9 steps with times … ], "model": "qwen2.5:3b"}}

direct call by another agent (no LLM):
{"user_id": "u1", "payload": {"action": "laundry.put", "args": {"pieces": "black t-shirt"}}}
```

8. Other team modules used

Branch	Used
`feat/bo8-orchestrator` (BO8)	Agent protocol `shared/agent_card.py`, card format, AGENTS.md rules (one tool per agent, next_actions).
`BO6/local_catalogue` (BO6)	**M7** code unchanged + its catalogue and FashionCLIP index → local_shops and gap_filling agents.
`garde-robe`, `feat/bo5-smartbuy`	Checked: M1 / M6 not runnable yet in our branch → their parts of "Should I buy", "Complete the look", "Declutter" are built in our agents (FashionCLIP + M2 + M3) and can switch to M1 / M6 later.

9. Ideas not done yet

Idea	Why / next step
Batch upload — multi-item detection (YOLO)	Needs M1's upload pipeline (BO3); a YOLO detector would feed M9 / M1.
Pinterest / Instagram import	Needs the platforms' APIs (OAuth, app review) and M1 / M5.

n8n: the agents themselves stay in Python (they need FashionCLIP, OutfitTransformer, Blender, and BO8's orchestrator is LangGraph / Python), but an n8n workflow is provided to show them from the outside — see section 12.

10. How to test

```
1. ollama serve                     (model: ollama pull qwen2.5:3b)
2. cd backend
   $env:M02_USE_MOCK="false"; $env:M03_USE_MOCK="false"; $env:M04_USE_MOCK="false"
   $env:M17_USE_MOCK="false"; $env:M09_USE_MOCK="false"; $env:M07_USE_MOCK="false"
   python -m uvicorn api.main:app --host 0.0.0.0 --port 8000
3. http://localhost:8000/bo/demo     choose the agent (★ = orchestrators) and the wardrobe, click a suggestion
                                      or type; the pipeline appears on the right; 📷 photo for "Should I buy?"
   http://localhost:8000/bo/screens  app-like buttons (no LLM) + diagram of who calls whom
4. http://localhost:8000/docs        sections "BO1 / BO2 agents" and "agents"
5. n8n: import backend/bo/n8n/dressme_agents.json, press Execute (section 12)
6. python -m pytest -q bo/tests shared/test_agents.py   (scripted LLM, no Ollama needed)
```

Test	Result
All 24 actions of the 16 idea agents, structured, real modules	24 / 24 ok (0.0 – 16 s each; capsule ILP is the slowest)
Routing of a 9-message conversation with the LLM (EN + FR, cross-BO)	9 / 9 routed to the right agent and action (17 s first message, 6–12 s next)
Automated tests (bo, agents, modules, stylist, orchestrator)	64 passed
Limits seen	3B model: answers sometimes add an alternative from an earlier turn; latency 6–20 s per request on a 4 GB GPU (a 7B model or a cloud LLM improves both).

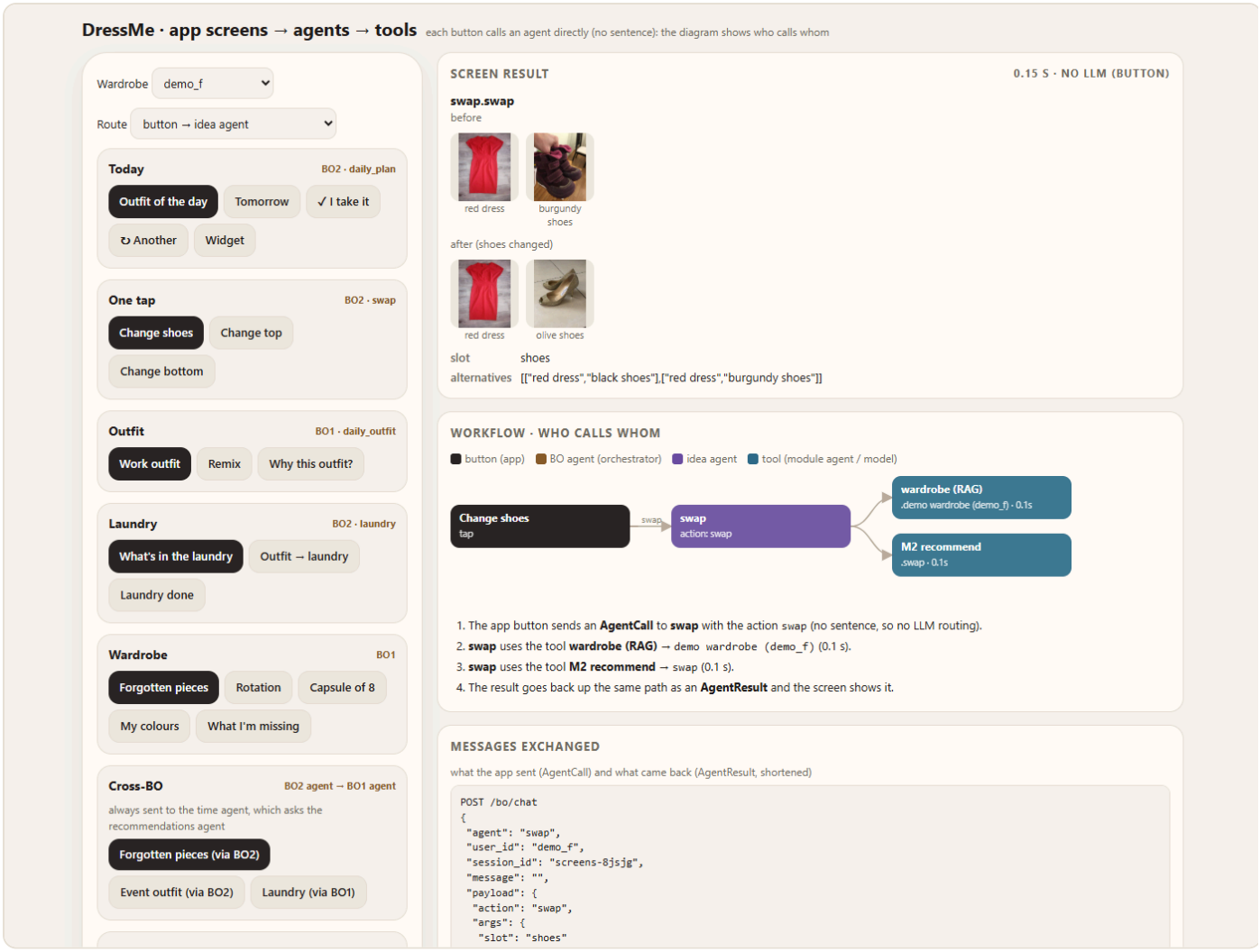
11. Buttons and agent workflow (how agents call each other and use tools)

In the real app most actions are **buttons**: the screen already knows what the user wants, so it sends a structured `AgentCall` with `payload.action` and `args`. No LLM is needed (0.05–1 s), and the same agents answer as in the chat. The chat (one sentence) only adds two LLM steps at the start: the BO agent picks the idea agent, then that agent picks the action.

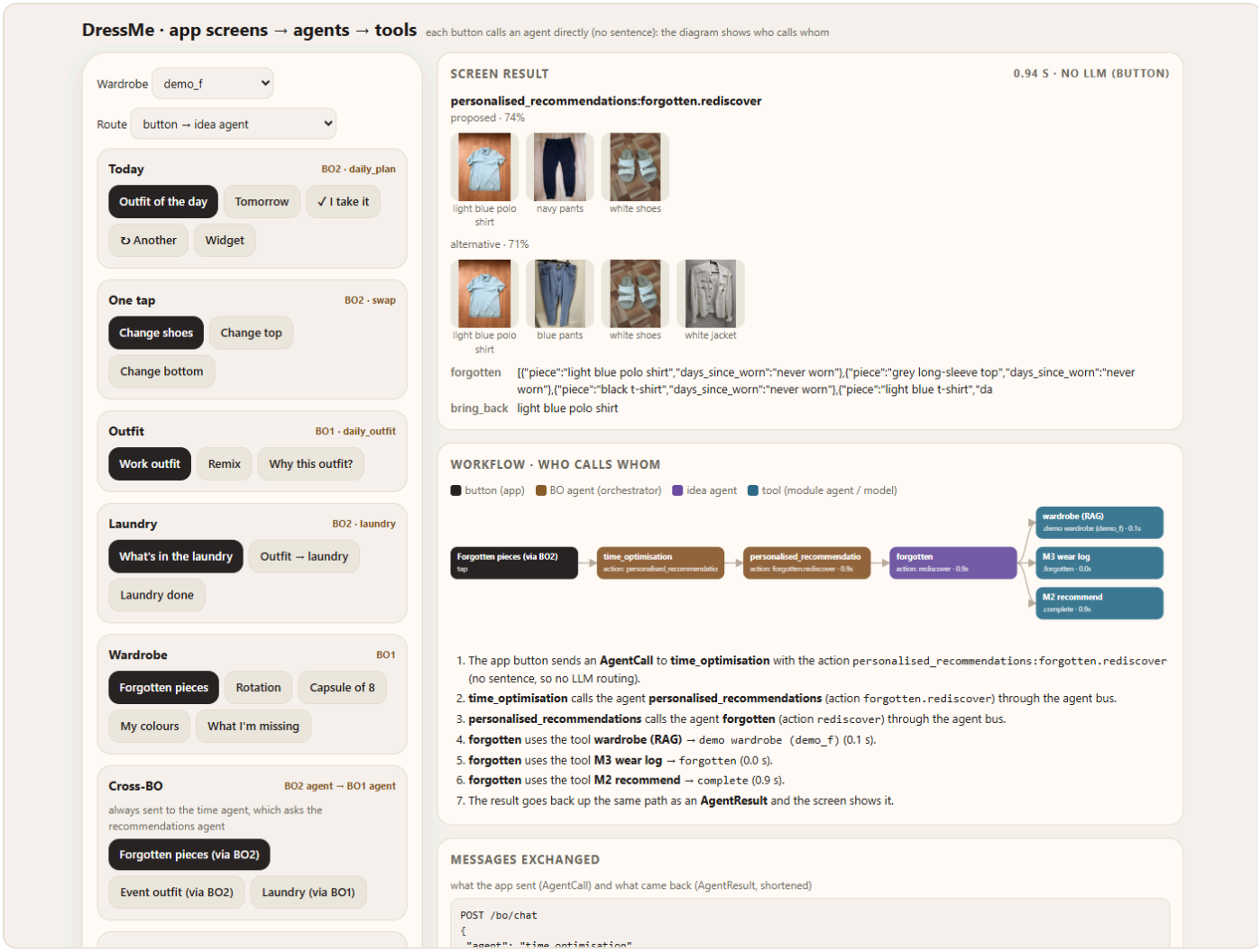
Button (screen)	AgentCall sent	Path (agents → tools)
Outfit of the day	`{"agent":"daily_plan","payload":{"action":"today"}}`	daily_plan → wardrobe (RAG) → M4 scheduler.today
Change shoes	`{"agent":"swap","payload":{"action":"swap","args":{"slot":"shoes"}}}`	swap → wardrobe → M2 recommend.swap
Outfit → laundry	`{"agent":"laundry","payload":{"action":"put","args":{"pieces":"red dress, olive shoes"}}}`	laundry → wardrobe → M3 wear log.status
Capsule of 8	`{"agent":"capsule","payload":{"action":"build","args":{"size":8}}}`	capsule → wardrobe → M17 capsule (ILP)
Pack for Paris · 3 days	`{"agent":"travel","payload":{"action":"plan","args":{"city":"Paris","days":3}}}`	travel → wardrobe → M2 recommend ×3 (one per day, with weather)
Shops: black trousers	`{"agent":"local_shops","payload":{"action":"find","args":{"query":"…"}}}`	local_shops → M7 local catalogue.search
Forgotten pieces (via BO2)	`{"agent":"time_optimisation","payload":{"action":"personalised_recommendations:forgotten.rediscover"}}`	time_optimisation → personalised_recommendations → forgotten → wardrobe, M3.forgotten, M2.complete

Rules of the workflow

- **Who can call whom.** App or BO8 → BO agent (orchestrator) → idea agent → tools. An idea agent may also call another idea agent (e.g. gap_filling → local_shops). Cross-BO calls always pass through the partner BO agent (`"partner:agent.action"`), never directly.
- **How they call.** Every call goes through the agent bus `ctx.call(agent, action, payload)` . Between agents the message is `AgentCall` → `AgentResult` ; for a tool (module agent M2/M3/M4/M7/M9/M17) it is `{action, payload}` → `{ok, result}` , validated with the module's request model.
- **How they use tools.** The idea agent's workflow decides which tools to call and in which order (e.g. forgotten: M3 for unworn pieces, then M2 to build an outfit around one). It reads the session memory (last outfit) for buttons like "Outfit → laundry" or "Another".
- **Trace.** Each call adds a step (agent / tool / memory / wardrobe, with time) and the sub-agent steps are nested, so the screen can draw the tree below.



/bo/screens — "Change shoes": button → swap agent → wardrobe + M2 (0.15 s, no LLM). Before/after pieces, the workflow tree and the AgentCall sent.



Cross-BO button: time_optimisation (BO2) → personalised_recommendations (BO1) → forgotten → wardrobe, M3, M2 (0.9 s).

12. n8n showcase: agents talking to each other, seen from outside

What n8n is here: n8n plays the role of an **external app or another BO's service**. It only sends `AgentCall`s over HTTP to `POST /bo/chat`, exactly as BO8's orchestrator or the mobile app would. The agents, their LLM and the tools are not in n8n: they run in the DressMe backend. This makes the agent pipeline visible as a diagram, and proves the agents are reachable by any outside system. File: `backend/bo/n8n/dressme_agents.json` (n8n 2.x, import → Execute).

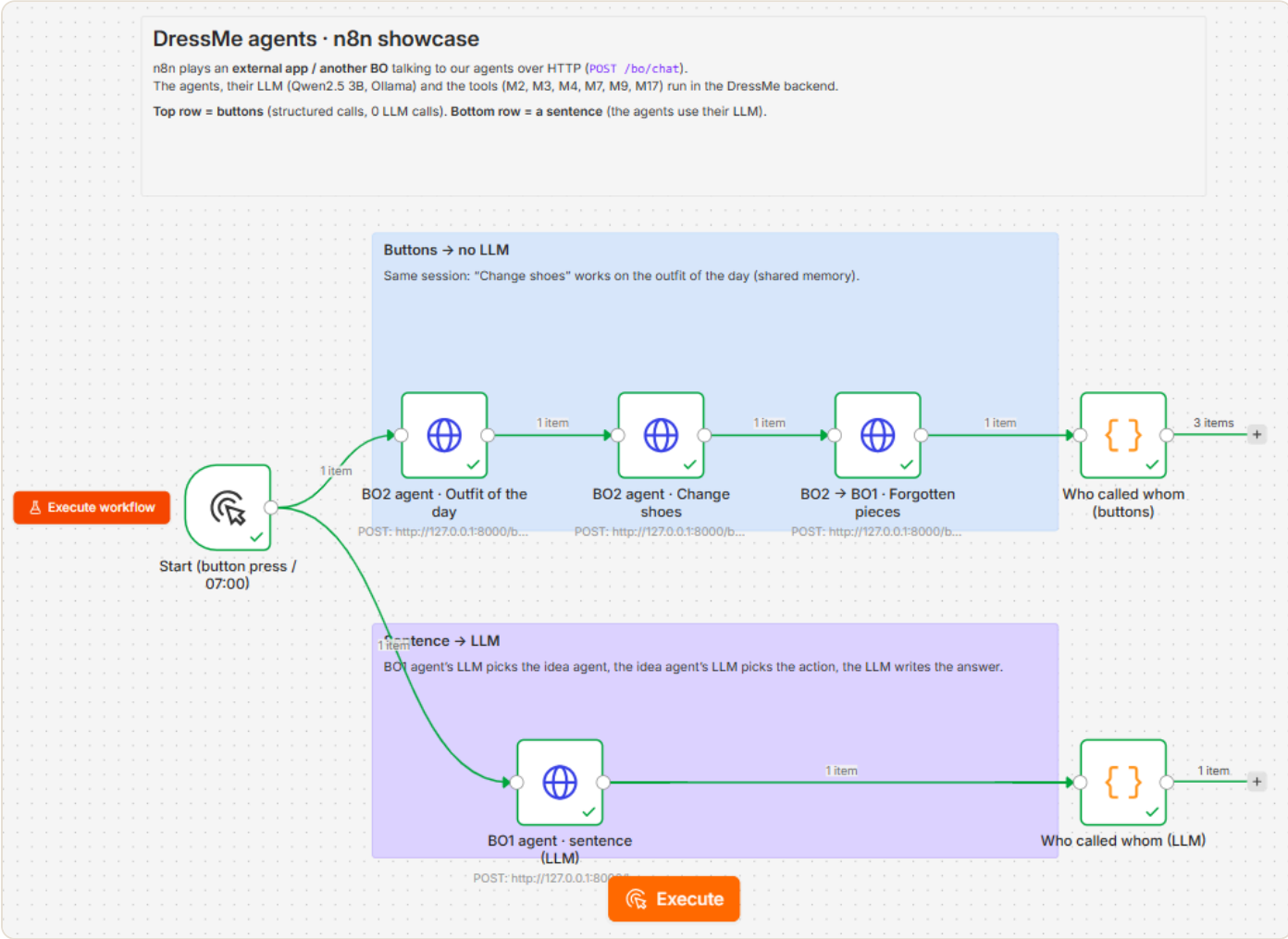


Figure 6 · The workflow after a real run (n8n 2.43, all nodes green). Top row = three button calls in the same session (no LLM). Bottom row = one sentence (LLM). The two "Who called whom" nodes turn each agent's `result.steps` into the call tree below.

```
Output of the run (node "Who called whom", copied from n8n):

== Outfit of the day (button)                    llm_calls: 0
app → time_optimisation
  time_optimisation → agent daily_plan.today
    daily_plan → tool m04_scheduler.today (2 ms)

== Change shoes (button, same session → works on the outfit above)   llm_calls: 0
app → time_optimisation
  time_optimisation → agent swap.swap
    swap → tool m02_recommendation.swap (88 ms)

== Forgotten pieces, BO2 → BO1 (button)          llm_calls: 0
app → time_optimisation
  time_optimisation → agent personalised_recommendations.forgotten.rediscover
    personalised_recommendations → agent forgotten.rediscover
      forgotten → tool m03_wear_log.forgotten (6 ms)
      forgotten → tool m02_recommendation.complete (834 ms)

== "What should I wear tomorrow for work?" (sentence)   llm_calls: 3
app → personalised_recommendations
  personalised_recommendations → LLM llm_route (7524 ms)      ← picks the idea agent
  personalised_recommendations → agent daily_outfit
    daily_outfit → LLM llm_route (2775 ms)                    ← picks the action + arguments
    daily_outfit → tool m04_scheduler.calendar (4 ms)
    daily_outfit → tool m02_recommendation.recommend (2535 ms)
  personalised_recommendations → LLM llm_answer (3896 ms)     ← writes the answer
reply: "For tomorrow, you should wear a black t-shirt, a blue skirt, and black shoes. This outfit was
recommended as it works well together and suits the formality level required for work. As an
alternative, you could also wear a light blue t-shirt, blue pants, and light blue shoes."
```

What the run shows

- Same agents, two ways: buttons → 0 LLM calls, under 1 s each; a sentence → 3 LLM calls (≈14 s) for the same kind of result.
- Agents call agents: BO2's agent hands the forgotten-pieces request to BO1's agent, which hands it to the forgotten agent: 3 agents, 2 tools.
- Shared memory between agents: "Change shoes" (swap, BO2) changes the outfit picked one call before by daily_plan, because both read the session blackboard.
- The LLM never picks the clothes: the tools (M2 / OutfitTransformer, M3, M4) do; the LLM routes and explains.